

Návrh a implementace databázových systémů

Procedurální rozšíření jazyka SQL,
procedurální funkce

Procedurální funkce

- funkce psané v nějakém procedurálním jazyce
 - použijeme PL/pgSQL, ale PostgreSQL umožňuje i Python, Perl nebo Tcl

```
CREATE FUNCTION func(parlist) RETURNS datatype AS $$  
BEGIN  
    ...  
END;  
$$ LANGUAGE plpgsql;
```

PL/pgSQL

- obdoba jazyka PL/SQL firmy ORACLE
- umožňuje
 - vytvářet funkce a obsluhu triggerů
 - přidat řízení běhu programu k jazyku SQL
 - provádět složité výpočty
 - využívat všech datových typů v DB
 - využívat operací a jazyka SQL
 - být důvěryhodný pro server

PL/pgSQL

- vstupní parametry

- jakýkoli datový typ, definovaný v DB, i tabulka
- odkazy číslováním (\$1, \$2, atd.)

`CREATE FUNCTION funkce(int, real) RETURNS ....`

- Ize ale v seznamu parametrů pojmenovat

`CREATE FUNCTION sales_tax(subtotal real) RETURNS ....`

- návratové hodnoty

- jakýkoli základní či kompozitní datový typ
- set (množina)
- tabulka
- void

PL/pgSQL

- blokově orientovaný jazyk, vnořování bloků
 - nezaměňovat BEGIN s transakcemi

- case insensitive

- blok má strukturu

- návěští (label) – nepovinné

- deklarace

- tělo bloku

- komentář

- --

- /* ... */

```
[ <<label>> ]  
[ DECLARE  
    declarations ]  
BEGIN  
    statements  
END [ label ];
```

Deklarace

- všechny proměnné musejí být deklarovány
 - kromě iterátoru LOOP a FOR
 - mohou mít libovolný datový typ, i z SQL
 - kopírování datových typů
 - `variable%TYPE`
 - `table_name%ROWTYPE`
 - syntaxe

`name [ CONSTANT ] type [ NOT NULL ] [ { DEFAULT | := } expression ];`

```
user_id  integer;    quantity  numeric(5);    url  varchar;
myrow    tablename%ROWTYPE;
myfield  tablename.columnname%TYPE;
arow     RECORD; -- obecný kompozitní typ bez určení struktury
```

Deklarace

■ příklady

```
CREATE FUNCTION SP(x int, y int, OUT sum int, OUT prod int) AS $$  
BEGIN  
    sum := x + y;  
    prod := x * y;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION ext_sales(n int)  
    RETURNS TABLE(quantity int, total numeric) AS $$  
BEGIN  
    RETURN QUERY SELECT quantity, quantity * price FROM  
sales    WHERE itemno = n;  
END; $$ LANGUAGE plpgsql;
```

Deklarace

■ příklady

```
CREATE FUNCTION merge_fields(t_row table1) RETURNS text AS $$  
DECLARE  
    t2_row table2%ROWTYPE;  
BEGIN  
    SELECT * INTO t2_row FROM table2 WHERE ... ;  
    RETURN t_row.f1 || t2_row.f3 || t_row.f5 || t2_row.f7;  
END;  
$$ LANGUAGE plpgsql;
```

```
SELECT merge_fields(t.*) FROM table1 t WHERE ... ;
```


Příkazy

- přiřazení

`variable := expression;`

- SQL příkazy nevracející data

`BEGIN`

`UPDATE mytab SET val = val + delta WHERE id =
key;`

`END;`

- dotazy vracející jeden řádek (do proměnné)

`SELECT select_expressions INTO target FROM ...;`

`INSERT ... RETURNING expressions INTO target;`

- nedělat nic: příkaz `NULL;`

- informace o zpracování příkazu

`GET DIAGNOSTICS variable = item [ , ... ];`

proměnná `FOUND`

Příkazy

- dynamické SQL příkazy
 - často chceme, aby SQL příkaz obsahoval pokaždé něco jiného (jiné identifikátory apod.)
 - příkaz **EXECUTE *command_string***
 - uvnitř příkazového řetězce nutno ošetřit uvozovky – např. funkcemi **quote_ident** a **quote_literal**

```
CREATE FUNCTION vypis(tab text, podm text) RETURNS void AS $$  
BEGIN  
    EXECUTE      'SELECT * FROM ' || quote_ident(tab) ||  
                ' WHERE ' || quote_literal(podm);  
END;  
$$ LANGUAGE plpgsql;
```

Řízení běhu programu

- návrat z funkce

`RETURN expression;` -- jednotlivá hodnota

- vracení složitějších dat

- v každé funkci, vracející SETOF, je zavedena proměnná pro výsledek, do níž se ukládají data
 - tato proměnná je aditivní – lze do ní postupně přidávat data
- příkaz `RETURN QUERY query;` připojí do návratové proměnné výsledek dotazu
- příkaz `RETURN NEXT expression;` připojí do návratové proměnné výsledek výrazu
- tyto příkazy neukončí funkci, to je nutno zařídit dalším samostatným příkazem `RETURN;`

Řízení běhu programu

■ příklad

```
CREATE FUNCTION getAllTbl() RETURNS SETOF Tbl AS $$
DECLARE
    r Tbl%rowtype;
BEGIN
    FOR r IN SELECT * FROM Tbl WHERE id > 0
    LOOP
        -- nějaké zpracování
        RETURN NEXT r; -- vrátí aktuální řádek SELECT
    END LOOP;
    RETURN;
END
$$ LANGUAGE plpgsql;

SELECT * FROM getAllTbl();
```

Řízení běhu programu

■ podmínka IF a CASE

IF ... THEN

IF ... THEN ... ELSE

IF ... THEN ... ELSIF ... THEN ... ELSE

CASE ... WHEN ... THEN ... ELSE ... END CASE -- rovnost

oper. CASE WHEN ... THEN ... ELSE ... END CASE -- větvení

IF number = 0 THEN

result := 'zero';

ELSIF number > 0 THEN

result := 'positive';

ELSIF number < 0 THEN

result := 'negative';

ELSE

result := 'NULL';

END IF;

Řízení běhu programu

■ podmínka IF a CASE

CASE x

WHEN 1, 2 THEN

msg := 'jedna nebo dvě';

ELSE

msg := 'něco jiného než jedna či dvě';

END CASE;

CASE

WHEN x BETWEEN 0 AND 10 THEN

msg := 'x je mezi 1 a 10';

WHEN x BETWEEN 11 AND 20 THEN

msg := 'x je mezi 11 a 20';

END CASE;

Řízení běhu programu

- cykly

- nekonečná smyčka LOOP (než přijde EXIT či RETURN)

LOOP

příkazy

END LOOP;

- podmíněná smyčka WHILE

WHILE podmínka LOOP

příkazy

END LOOP;

- klíčová slova EXIT a CONTINUE

- EXIT ukončí cyklus, CONTINUE skočí na další iteraci
- nepodmíněně: EXIT; CONTINUE;
- podmíněně: EXIT WHEN ... ; CONTINUE WHEN ...;

Řízení běhu programu

- cykly

- iterační smyčka FOR

```
FOR i IN 1..10 LOOP
```

```
-- i bude postupně 1,2,3,4,5,6,7,8,9,10
```

```
END LOOP;
```

```
-----
```

```
FOR i IN REVERSE 10..1 LOOP
```

```
-- i bude postupně 10,9,8,7,6,5,4,3,2,1
```

```
END LOOP;
```

```
-----
```

```
FOR i IN REVERSE 10..1 BY 2 LOOP
```

```
-- i bude postupně 10,8,6,4,2
```

```
END LOOP;
```


Řízení běhu programu

- procházení výsledků dotazů
 - „procházecí“ smyčka FOR

```
CREATE FUNCTION cs_refresh_mviews() RETURNS integer AS $$
DECLARE
    mviews cs_mviews%rowtype;
BEGIN
    FOR mviews IN SELECT * FROM cs_mviews ORDER BY
key
    LOOP
        -- "mviews" obsahuje jeden řádek z dotazu
    END LOOP;
    RETURN 1;
END;
$$ LANGUAGE plpgsql;
```

Ošetření chyb

- standardně chyba ukončí provádění funkce
- lze ošetřit výjimkou

```
BEGIN
```

```
    statements
```

```
EXCEPTION
```

```
    WHEN condition [ OR condition ... ] THEN
```

```
        handler_statements
```

```
    [ WHEN condition [ OR condition ... ] THEN
```

```
        handler_statements
```

```
    ... ]
```

```
END;
```

Ošetření chyb

- výjimka
 - stav proměnných je zachován
 - ukončí aktuální transakci a provede ROLLBACK
 - příklad: vrátí hodnotu $x+1$, ale UPDATE neproběhne

BEGIN

UPDATE mytab SET name = 'Joe' WHERE lastname = 'Jones';

x := x + 1;

y := x / 0;

EXCEPTION

WHEN division_by_zero THEN

RAISE NOTICE 'caught division_by_zero';

RETURN x;

END;

Ošetření chyb

- příkaz RAISE
 - různé úrovně priority, nejvyšší je EXCEPTION
 - EXCEPTION zastaví vykonávání akt. transakce
 - lze použít k ladicím výpisům (úroveň DEBUG)

```
RAISE NOTICE 'Calling cs_create_job';
```

```
RAISE 'Duplicate user ID: %', user_id USING ERRCODE =  
'unique_violation';
```